

EECS 3311 Course Project - Stage 1 Design Report

AI Cycling Training Coach

Project Overview

1.1 Project Description

Problem:

As sports training becomes more science-based, cycling training has also become more structured and data-driven. Cycling includes many different types of training, each designed to improve different aspects of performance. It is impossible to fit all these types of training into a cyclist's plan, and without a professional, it is also difficult to know which type of training is needed most.

One common measurement used in cycling is Functional Threshold Power (FTP). FTP is an estimate of the highest average power, measured in watts, that a cyclist can sustain for about one hour. It is commonly used as a benchmark for cycling fitness and data-driven training.

Recreational and amateur cyclists have limited time and energy to dedicate to training, and many are unsure how to train effectively, what type of workout to perform, when to recover, or how to fit structured training into their daily lives. Although cycling training information is available from many sources, interpreting that information and turning it into a plan is difficult without access to a personal coach.

The AI Cycling Training Coach is intended to help cyclists organize their training by analyzing the rider's performance data, health data, training history, goals, and weekly schedule. This system will provide tailored plans for cyclists, suggestions the best workouts to improve, and explain questions that the cyclist might have about their training.

Target Users:

Target users are recreational and amateur cyclists who have a clear training goal, such as improving fitness, or becoming healthier. These users are expected to

record training data with devices such as bike computers, heart rate monitor, or power meter.

Agent Capabilities:

The AI Cycling Training Coach will analyze a rider's training data, fitness data, goals, and weekly schedule to provide personalized training guidance. The agent can generate training plans, recommend workouts, analyze completed rides, identify areas for improvement, suggest recovery when needed, and answer natural-language questions about the rider's training. The agent can also adjust future recommendations based on the rider's needs.

Why an AI Agent is Appropriate:

An AI agent is appropriate for this project because there are many factors that can affect a cyclist's training, such as their health, fitness, fatigue, and how much time they have available. A program based on fix rules is not enough to look at all these factors of the rider's situations. The AI agent can interpret all the rider's data, answer the rider's questions, explain training workouts, and adjust training to the rider's progress and needs.

Planned AI/LLM Model:

This project plans to use a DeepSeek LLM through the DeepSeek API as the AI model.

AI Interaction with the Software System:

The application will send relevant rider data and questions to the DeepSeek LLM through its API, and the model's response will be returned to the application and shown to the user.

1.2 Feature Specification

Feature ID and name:

F01 – Rider Profile Setup

Description: Allows the cyclist to create a profile that stores information needed for personalized training.

User Interaction: The user enters or edits the information through a profile page.

Input: Weight, FTP, maximum heart rate, resting heart rate, available training days, target weight, target FTP, main goal (short text).

Output: Saved rider profile.

AI Involvement: Deterministic.

Workflow: The cyclist enters the information and saves the profile.

Error Case: Invalid or missing values are rejected.

F02 – Ride Data Import

Description: Allows rider to upload a .csv file from a completed ride.

User Interaction: the user uploads a .csv file through an upload page.

Input: CSV file.

Output: store data such as date, duration, power, heart rate, and cadence.

AI Involvement: Deterministic.

Workflow: The cyclist uploads the file, the system parses it, and the data is saved in the rider's history.

Error Case: if the uploaded file is invalid, missing required data, or cannot be read, the system shows an error message.

F03 – AI Ride Analysis

Description: Analyze one completed ride and provide a summary of the rider's performance.

User Interaction: the user selects a ride from their ride history and clicks analyze.

Input: Data from ride history and user profile.

Output: AI-generated ride summary and observations.

AI Involvement: Hybrid.

Workflow: The system retrieves the data and sends the information to LLM. The LLM analyzes the ride and returns a summary.

Error Case: If the ride data is missing or incomplete, the system shows an error message.

F04 – Weekly Training Plan Generation

Description: Use LLM and rider's information to generate a weekly training plan.

User Interaction: The user clicks generate button.

Input: Rider's profile and rider's ride history.

Output: A weekly training plan with recommended workouts and recovery for each training day.

AI Involvement: Hybrid

Workflow: The system collects the rider's information and sends it to LLM. The LLM returns a weekly training plan, and the system displays it to the user.

Error Case: If the ride data is missing or incomplete, the system shows an error message.

F05 – AI Chat

Description: Allows the rider to ask questions about their training.

User Interaction: The user types a question into a chat page.

Input: Rider's question, rider's profile, and ride history.

Output: AI-generated answer.

AI Involvement: Hybrid.

Workflow: The system receives the rider's questions, collects recent ride history, sends them to the LLM, and displays the response.

Error Case: If the LLM does not respond or data is unavailable, the system shows an error message.

F06 – Progress Tracking

Description: Shows how the rider's performance changes over time and comparison to their goal.

User Interaction: the user opens a progress page.

Input: Saved FTP, weight, ride history, and goals.

Output: Progress charts or trend information over time.

AI Involvement: Deterministic.

Workflow: The system reads the rider's saved data and displays changes over time.

Error Case: if there is not enough data, the system informs the user.

F07 – Weakness/Improvement Detection

Description: Uses the user's training history to identify areas of improvement

User Interaction: The user clicks an Analyze Progress button.

Input: Rider's profile and ride history.

Output: AI-generated feedback for improvement.

AI Involvement: Hybrid.

Workflow: The system uses AIChatService with a predefined prompt. The LLM analyzes the data and respond.

Error Case: If there is not enough training data, the system tells the user that more data is needed.

F08 – Weekly Training Summary

Description: Provides a summary of the rider's ride history of the past week.

User Interaction: the user opens the weekly summary page.

Input: Rider's profile and ride history.

Output: Weekly training summary.

AI Involvement: Hybrid.

Workflow: The system uses AIChatService with a predefined prompt. The LLM analyzes the data and respond.

Error Case: If there are no rides in the past week, the system will show an error message.

F09 – Ride Comparison

Description: Compares the data from two rides in the ride history.

User Interaction: the user clicks the compare button in ride history and then selects another ride in the ride history.

Input: Ride history.

Output: Comparison of values such as duration, power, heart rate and cadence.

AI Involvement: Deterministic.

Workflow: The system collects the two selected rides, compares their data, and displays the difference.

Error Case: if there is missing data from the ride, the system shows an error message.

F10 – Workout Calendar

Description: Displays the rider's planned and completed workouts on a calendar.

User Interaction: the user opens the calendar page.

Input: Weekly training plan and completed ride history.

Output: Display a calendar of planned workouts and completed rides.

AI Involvement: Deterministic.

Workflow: The system collects the rider's weekly training plan and completed rides, then display them in a calendar.

Error Case: If there are no completed rides or plans, the calendar is empty.

F11 – Personal Best

Description: Displays the rider's best performance values from ride history in the past month.

User Interaction: the user opens the personal best page.

Input: Ride History.

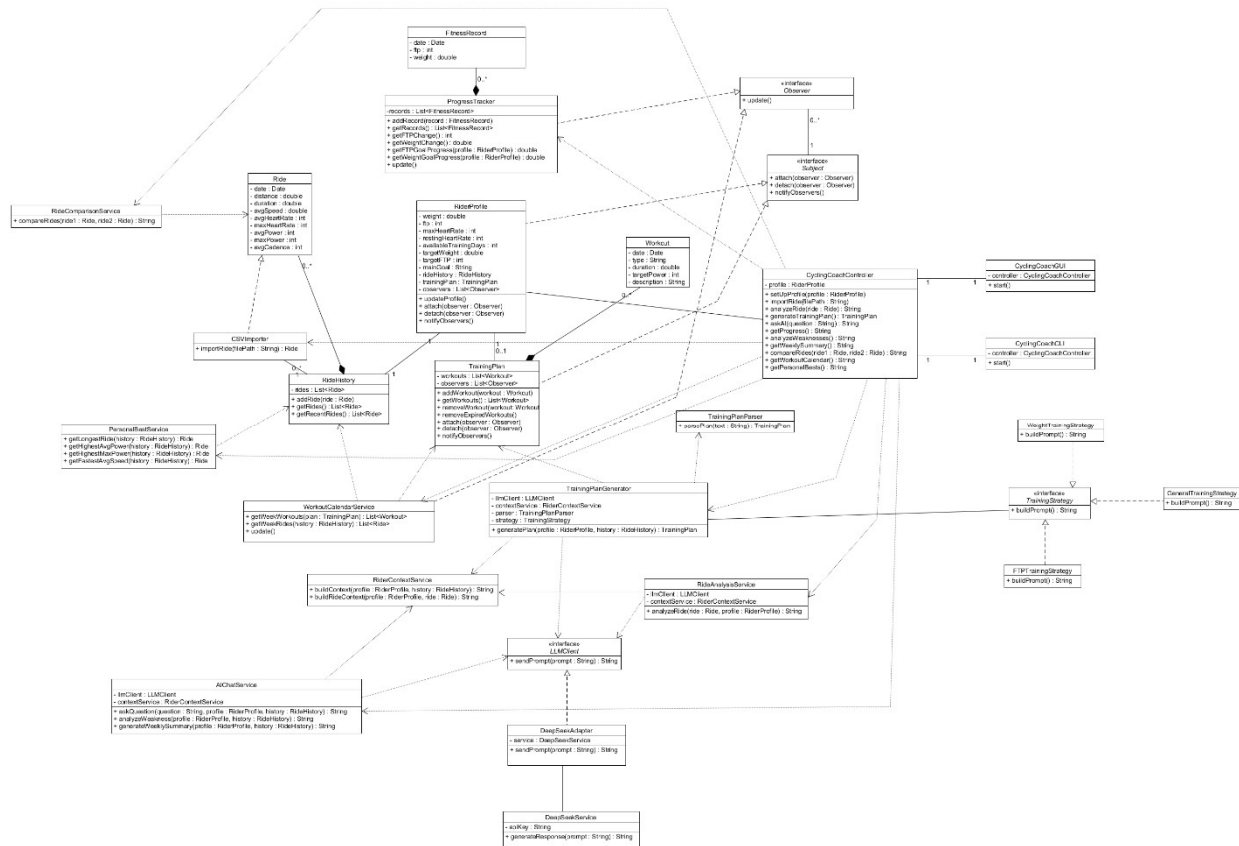
Output: Best values such as longest distance, highest average power, highest maximum power, and fastest average speed.

AI Involvement: Deterministic.

Workflow: The System compares the rider's stored rides and finds the best values for each category.

Error Case: If there are no completed rides, the system tells the user that no personal bests are available.

2.1 Class Diagram



2.1.1 Adapter Pattern

This pattern is used to connect the application with the DeepSeek service without making the rest of the system dependent on DeepSeek. With the use of the adapter DeepSeekAdapter, DeepSeek specific code is hidden from classes such as AIChatService, RideAnalysisService, and TrainingPlanGenerator. LLMClient is the target interface, DeepSeekAdaptor is the adapter, and DeepSeekService is the adaptee that contains the DeepSeek implementation. The adapter allows the code to be more flexible and easier to maintain because the rest of the system only depends on the LLMClient interface. If another LLM provider is used in the future, a new adapter can be created without changing the other AI-related classes.

2.1.2 Façade Pattern

The Façade Pattern is used with the controller. It allows the GUI and CLI to have a single point of entry to the application and its features. The GUI and CLI communicate with CyclingCoachController instead of directly communicating with backend classes such as CSVImporter, AIChatService, and other service classes.

CyclingCoachController acts as the façade, while the backend classes form the subsystem. This pattern reduces the amount of backend detail that the GUI and CLI need to know and makes the system easier to maintain and modify.

2.1.3 Observer Pattern

The Observer Pattern is used to keep related data automatically updated. In this project, RiderProfile and TrainingPlan act as subjects. Changes to them can automatically notify ProgressTracker and WorkoutCalendarService, which act as observers. This allows related data to be updated when changes occur. The pattern also reduces coupling because the subjects do not need to know the classes that are observing them. They only communicate through the Observer interface, which makes it easier to add or remove observers later.

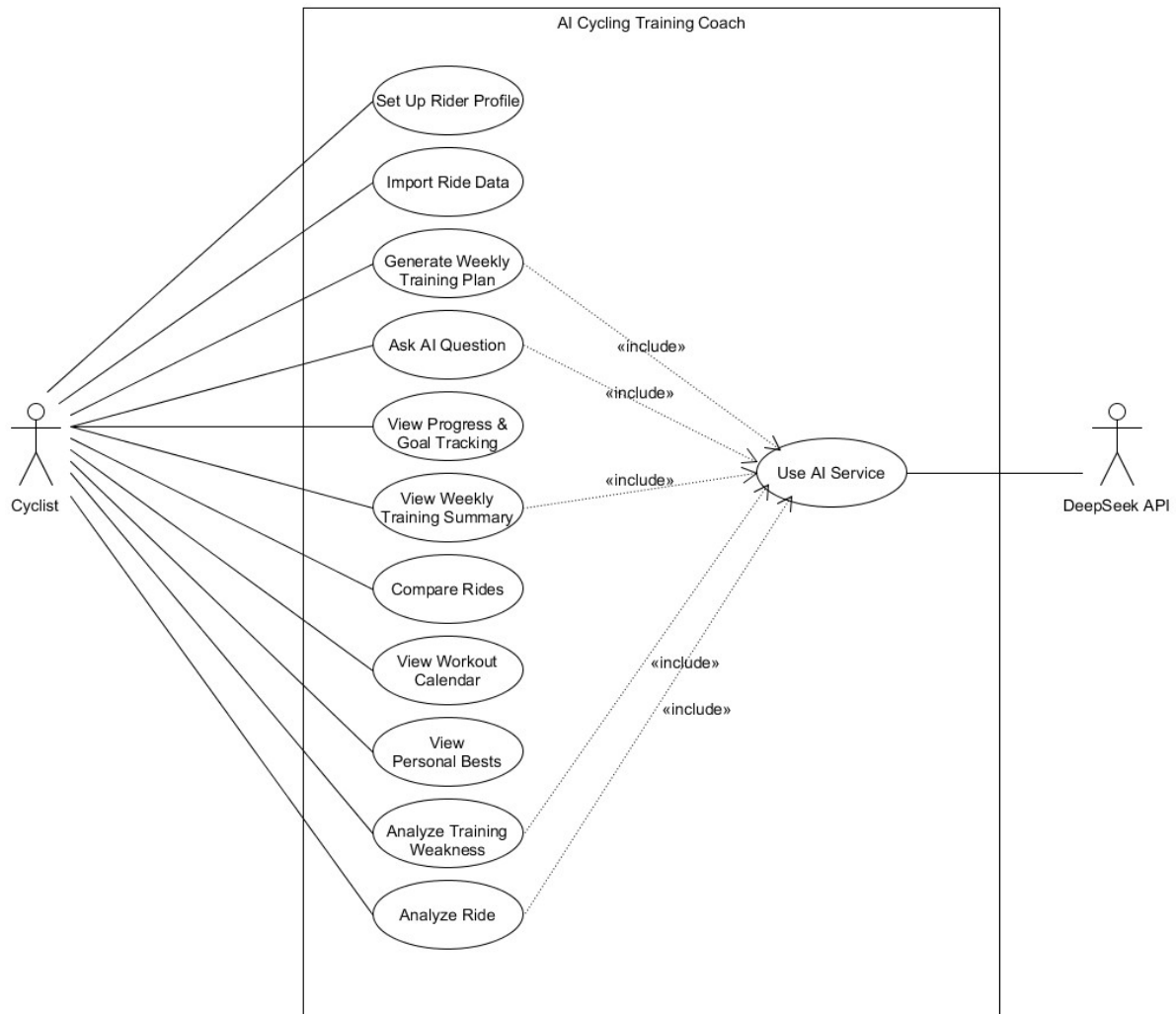
2.1.4 MVC Pattern

This pattern is used to separate the user interface from the backend classes and logic. CyclingCoachGUI and CyclingCoachCLI act as the View, CyclingCoachController acts as the Controller, and backend classes such as AIChatService and RideHistory make up the Model. By separating the system into these parts, it allows the GUI and CLI to change without requiring major changes to the backend logic. It also allows both interfaces to reuse the same controller and model classes, which reduces duplication and makes the system easier to maintain.

2.1.5 Strategy Pattern

The strategy pattern is used to change how weekly training plan is generated based on what the main goal of the rider is. TrainingStrategy is the strategy interface, and FTPTrainingStrategy, WeightTrainingStrategy, and GeneralTrainingStrategy is the concret strategies. When a training plan is generated, the prompt will differ base on the rider's main goal. This pattern makes the training plan generation more flexible and easy to extend.

2.2 Use Case Description



UC01 – Set Up Rider Profile

Actor: Cyclist

Goal: Create or update the rider's profile

Preconditions: the application is running

Trigger: The rider selects rider profile setup

Main Success Scenario:

1. The rider opens rider profile
2. The rider enters all data
3. The rider select save

4. The GUI sends the profile to CyclingCoachController
5. The controller updates RiderProfile
6. RiderProfile notifies its observer of the change
7. The system confirm that the profile is saved

Exception:

1. If not all data is filled in, the system display an error message
2. If a value enter is invalid, the system rejects and ask for correction

Postconditions: The rider profile contains the updated rider information.

Related Feature: F01 – Rider Profile Setup

UC02 – Import Ride Data

Actor: Cyclist

Goal: Import a completed ride data from a CSV file to RideHistory

Preconditions: the application is running and csv File is valid and formatted correctly.

Trigger: The rider selects upload ride.

Main Success Scenario:

1. The rider selects a csv file
2. The GUI sends the filepath to CyclingCoachController
3. The controllers uses CSVImporter
4. CSVImporter reads the csv file and create Ride object
5. The controller adds the new Ride object to RideHistory
6. The system confirm that the ride is imported

Exception:

1. If file is missing or cannot be opened, the system display an error
2. If csv format is invalid or data is missing, the system rejects the file
3. If the ride cannot be parsed correctly, no ride is added to RideHistory

Postconditions: The ride is stored in RideHistory

Related Feature: F02 – Ride Data Import

UC03 – Analyze Ride

Actor: Cyclist

Goal: Analyze a selected completed ride from RideHistory and receive an AI-generated summary

Preconditions: the application is running, the rider have profile, and one completed ride in RideHistory

Trigger: The rider selects a ride and choose the Analyze Ride Option

Main Success Scenario:

1. The rider selects a completed ride from RideHistory
2. The GUI sends the selected ride to CyclingCoachController
3. The controller calls RideAnalysisService
4. RideAnalysisService use RideContextService to prepare rider and ride's information
5. The returned prompt is sent to DeepSeekAdaptor then to DeepSeekService
6. DeepSeek generates a response
7. The response is returned to the controller
8. The GUI displays the response

Exception:

1. If no ride is selected, the system ask the rider to select a ride
2. If the AI service is unavailable, the system display an error message
3. If the ride data is incomplete, the system informs rider that the ride cannot be analyzed.

Postconditions: The cyclist receives an AI generated response of the ride

Related Feature: F03 – Rider Profile Setup

UC04 – Generate Weekly Training Plan

Actor: Cyclist

Goal: generate a weekly training plan

Preconditions: the application is running, and rider profile is created

Trigger: The rider selects generate a weekly training plan

Main Success Scenario:

1. The GUI sends the request to CyclingCoachController
2. The controller calls TrainingPlanGenerator

3. TrainingPlanGenerator checks the rider's main goal, choose the right TrainingStrategy, return prompt
4. RiderContextService turns rider's profile and ride history into another prompt, and return
5. Combine both prompt and it is send through DeepSeekAdaptor to DeepSeekService
6. DeepSeek returns a generated weekly training plan
7. TrainingPlanParser converts the response into a TraingingPlan object
8. TrainingPlanGenerator notifies its observers that the plan has changed
9. WorkoutCalendarService receives the update and refresh the calendar
10. The training plan is returned to the controller.
11. The GUI displays the generated training plan

Exception:

1. If rider profile is incomplete, the system ask the cyclist to complete the missing information
2. If DeepSeek is unavailable, the system displays an error
3. If the generated response cannot be parsed into a valid training plan, the system reports the error and does not save the plan

Postconditions: a new weekly training plan is available for the cyclist

Related Feature: F04 – Weekly Training Plan Generation

UC05 – Ask AI Question

Actor: Cyclist

Goal: allows the cyclist to ask AI question and get an AI response based on rider's information

Preconditions: the application is running, and rider profile is complete

Trigger: The rider types a question in the AI chat and sends it

Main Success Scenario:

1. The cyclist enters a question in the AI chat
2. The GUI sends the question to CyclingCoachController
3. The controller calls AIChatService
4. AIChatService uses RiderContextService to prepare rider profile and ride history

5. The question and rider context is sent through DeepSeekAdaptor to DeepSeekService
6. DeepSeek returns an AI generated response
7. The response is returned through AIChatService and CyclingCoachController
8. The Gui display the response to the cyclist

Exception:

1. If the question is empty, the system ask the rider to enter a question
2. If AI service is unavailable, the system display an error message
3. If rider's information is unavailable, the system ask the cyclist to complete the information

Postconditions: The cyclist receives a response to their question

Related Feature: F05 – AI Chat

UC06 – View Progress & Goal Tracking

Actor: Cyclist

Goal: shows the cyclist changes in FTP and weight and see progress toward target FTP and target weight

Preconditions: the application is running, rider profile exist, ProgressTracker is not empty

Trigger: The rider selects the progress & goal option

Main Success Scenario:

1. Cyclist select progress & goal option
2. The GUI sends the request to CyclingCoachController
3. The controller request progress information from ProgressTracker
4. ProgressTracker calculates FTP change and weight change
5. ProgressTracker compares the current value with target values that is stored in rider's profile
6. The calculated information is returned to the controller
7. The GUI displays the information for the cyclist

Exception:

1. If rider's profile is missing values, the system asks the cyclist to complete

2. If there is not enough fitness records, the system display the current values instead

Postconditions: The rider can value their progression towards the selected goal

Related Feature: F06 – Progress Tracking

UC07 – Analyze Weakness

Actor: Cyclist

Goal: Return an AI response of the cyclist's weakness based on their information

Preconditions: the application is running, and rider's profile is complete

Trigger: The rider selects analyze weakness

Main Success Scenario:

1. The cyclist select analyze weakness
2. The GUI sends the request to CyclingCoachController
3. The controller calls AIChatService and runs analyzeWeakness()
4. AIChatService uses RiderContextService to prepare the rider profile and ride history information.
5. The rider's context is combine with predefined prompt for weakness analysis
6. The prompt goes through DeepSeekAdaptor to DeepSeekService
7. DeepSeek returns a response
8. The response is returned to the controller
9. The GUI displays the response to the cyclist

Exception:

1. If rider's profile is incomplete, the system ask the rider to complete it
2. If AI service is unavailable, the system displays an error message

Postconditions: The rider receive an AI generated response on their weakness

Related Feature: F07 – Weakness/Improvement Dectection

UC08 – View Weekly Training Summary

Actor: Cyclist

Goal: Provides the cyclist with a summary of their past week of training

Preconditions: the application is running, the rider's profile is complete, and RideHistory contains ride data from the recent week

Trigger: The rider selects weekly training summary option

Main Success Scenario:

1. The cyclist select weekly training summary option
2. The GUI sends the request to CyclingCoachController
3. The controller calls AIChatService.generateWeeklySummary()
4. AIChatService calls RiderContextService to prepare rider's information
5. The information is combined with preexisting prompt
6. The prompt is sent through DeepSeekAdaptor to DeepSeekService
7. DeepSeek returns a summary
8. The summary is returned back to the controller
9. The GUI displays the summary for the cyclist

Exception:

1. If there is no ride data from the recent week, the system informs the cyclist that there is not enough data
2. If AI service is unavailable, then system displays an error message

Postconditions: The rider receives an summary based on their recent week of performance

Related Feature: F08 – Weekly Training Summary

UC09 – Compare Rides

Actor: Cyclist

Goal: Compare two completed ride from RideHistory

Preconditions: at least two rides are stored in RideHistory

Trigger: The rider selects one ride, choose compare, then chooses another

Main Success Scenario:

1. The rider opens the ride history
2. The rider selects the first ride
3. The rider selects compare option
4. The rider selects another ride

5. The GUI or CLI sends both selected ride object to CyclingCoachController
6. The controller calls RideComparisonService
7. RideComparisonService compares values such as distance, duration, average speed, average heart rate, and average power.
8. The comparison result is returned to the controller
9. The GUI or CLI displays the comparison to the cyclist

Exception:

1. If fewer than two ride is available in RideHistory, the system tell cyclist that two rides is required
2. If cyclist selects the same ride twice, the system ask the cyclist to select a different second ride
3. If some ride data is missing, the system compares the available values

Postconditions: The cyclist receives a comparison of the two selected ride

Related Feature: F09 – Ride Comparison

UC10 – View Workout Calendar

Actor: Cyclist

Goal: allows the cyclist to see planned workouts and past rides in one calendar view

Preconditions: the application is running, the rider's profile is complete, and A TrainingPlan and/or completed ride is available

Trigger: The rider selects the workout calendar option

Main Success Scenario:

1. The cyclist opens the workout calendar
2. The GUI sends the request to CyclingCoachController
3. The controller calls WorkoutCalendarService
4. WorkoutCalendarService retrieves planned workout and completed rides from TrainingPlan and RideHistory
5. The planned workouts and completed rides are combined into the calendar view
6. The GUI display the calendar to the cyclist

Exception:

1. If no training plan exist, the calendar shows only completed rides

2. If no completed rides exist, the calendar shows only planned workout
3. If there are no workouts or rides, the system display an empty calendar

Postconditions: The rider can view a calendar of their rides and planned workouts

Related Feature: F10 – Workout Calendar

UC11 – View Personal Bests

Actor: Cyclist

Goal: allows cyclist to view their best performance from their completed rides.

Preconditions: the application is running, and at least one ride in RideHistory

Trigger: The rider selects personal bests option

Main Success Scenario:

1. The cyclist selects personal best option
2. The GUI sends the request to CyclingCoachController
3. The controller calls PersonalBestService
4. PersonalBestService reads the rides stored in RideHistory
5. The service identifies records such as longest distance, highest average power, highest maximum power, and fastest average speed
6. The personal best results are returned to the controller
7. The GUI displays the results to the cyclist

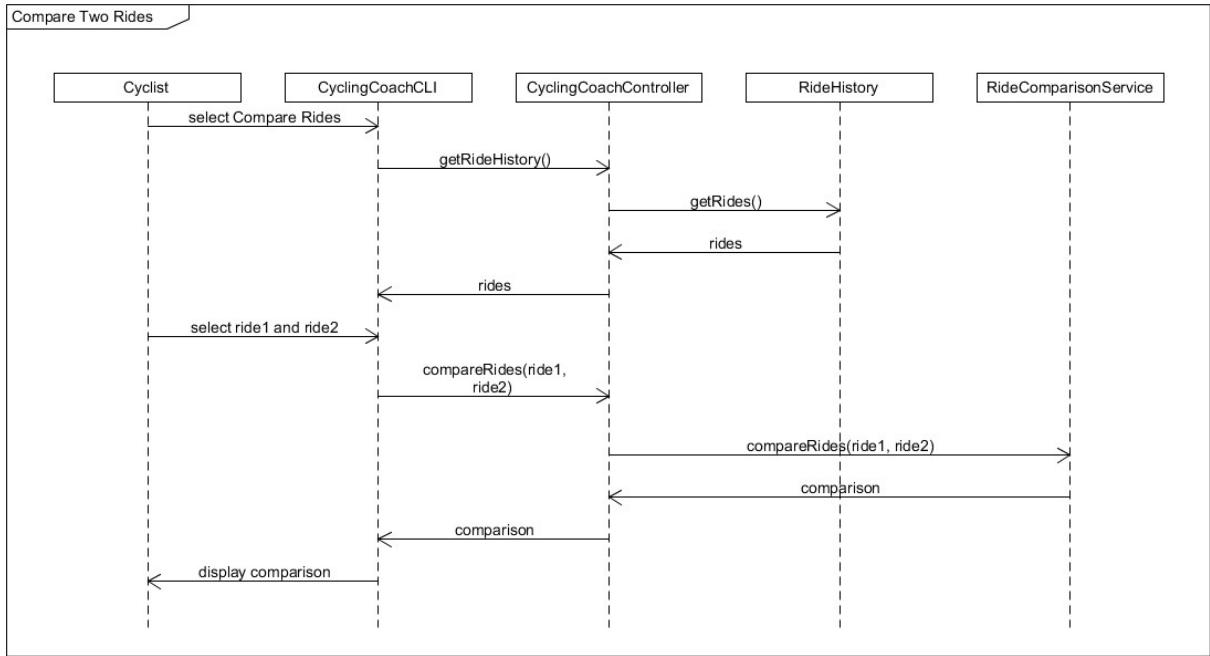
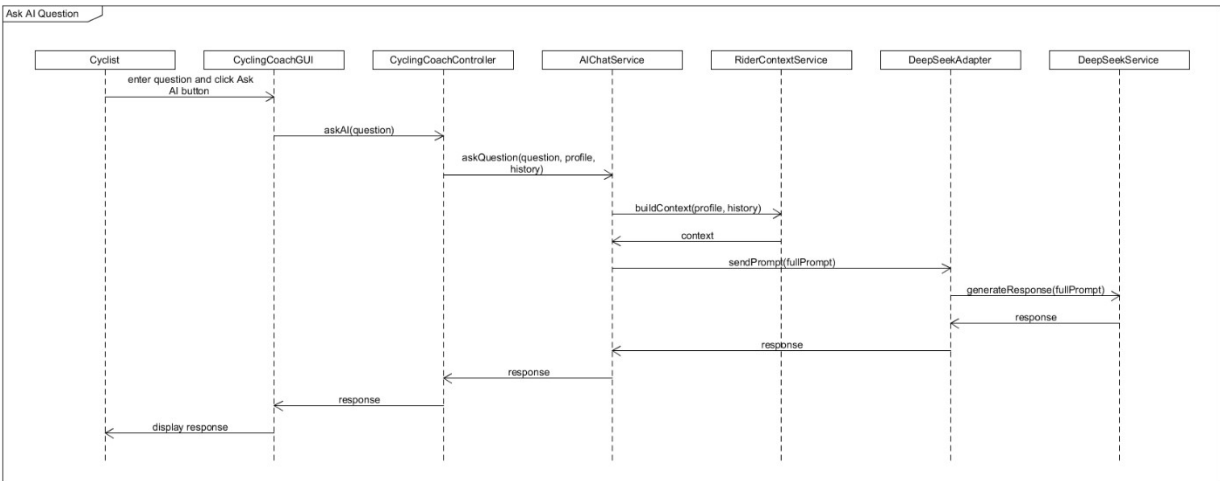
Exception:

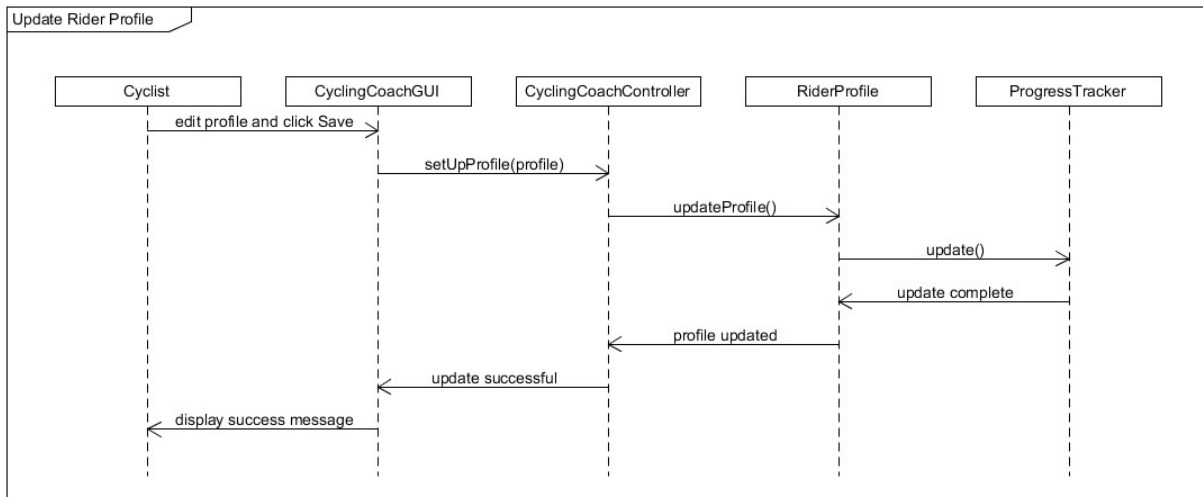
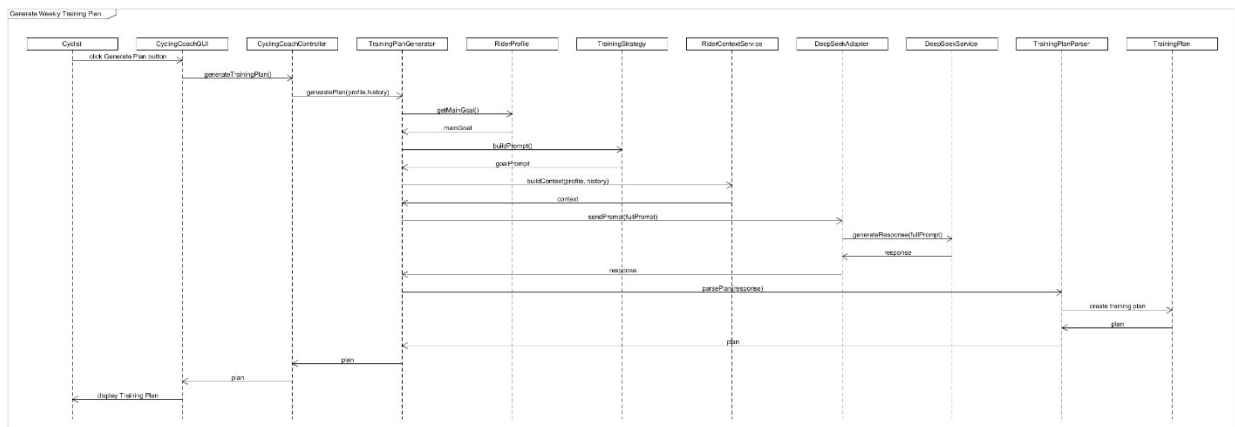
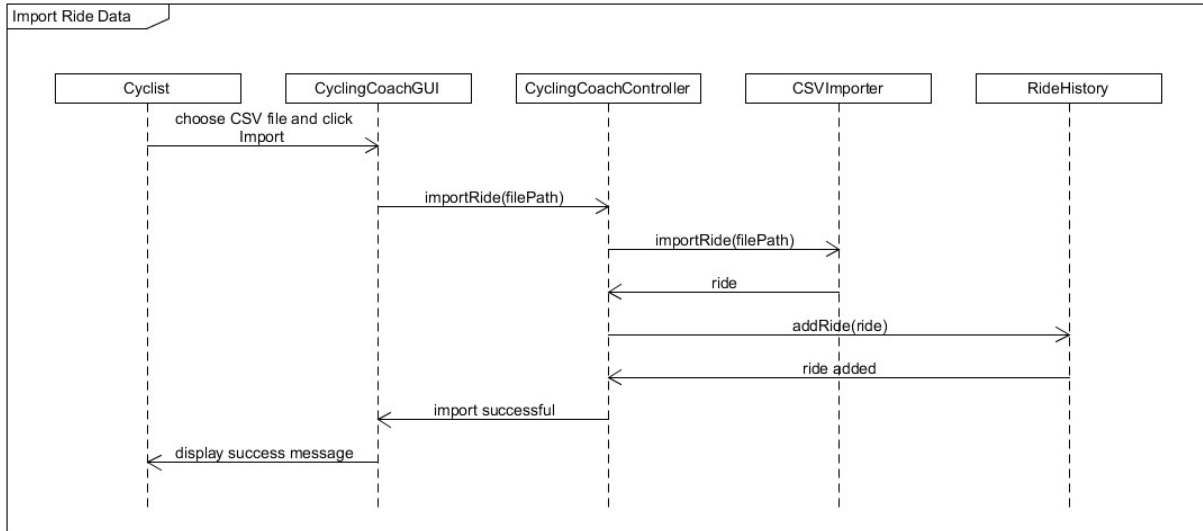
3. If there is no ride in RideHistory, the system informs the rider that no personal best is available
4. If some ride data is missing, the system only return records of data that is available

Postconditions: The rider receives the personal best results

Related Feature: F11 – Personal Best Detection

2.3 Sequence Diagram





3 Feature to Design Traceability Table

Feature	Description	Type	Use Case	Main Classes	Key Methods	Sequence Diagram	Design Pattern
F01	Rider Profile Setup	Deterministic	UC01 Set Up Rider Profile	CyclingCoachGUI, CyclingCoachController, RiderProfile, ProgressTracker	setUpProfile(), updateProfile(), notifyObservers(), update()	SD04 Update Rider Profile	MVC, Facade, Observer
F02	Ride Data Import	Deterministic	UC02 Import Ride Data	CyclingCoachGUI, CyclingCoachController, CSVImporter, RideHistory	importRide(), addRide()	SD03 Import Ride Data	MVC, Facade
F03	AI Ride Analysis	Hybrid	UC03 Analyze Ride	CyclingCoachGUI, CyclingCoachController, RideAnalysisService, RiderContextService, DeepSeekAdapter, DeepSeekService	analyzeRide(), buildRideContext(), sendPrompt(), generateResponse()	no SD	MVC, Facade, Adapter
F04	Weekly Training Plan Generation	Hybrid	UC04 Generate Weekly Training Plan	CyclingCoachGUI, CyclingCoachController, TrainingPlanGenerator, TrainingStrategy, RiderContextService, DeepSeekAdapter, DeepSeekService, TrainingPlanParser, TrainingPlan	generateTrainingPlan(), generatePlan(), buildPrompt(), buildContext(), sendPrompt(), generateResponse(), parsePlan()	SD01 Generate Weekly Training Plan	MVC, Facade, Strategy, Adapter, Observer
F05	AI Chat	Hybrid	UC05 Ask AI Question	CyclingCoachGUI, CyclingCoachController, AIChatService, RiderContextService, DeepSeekAdapter, DeepSeekService	askAI(), askQuestion(), buildContext(), sendPrompt(), generateResponse()	SD02 Ask AI Question	MVC, Facade, Adapter
F06	Progress Tracking	Deterministic	UC06 View Progress & Goal Tracking	CyclingCoachController, ProgressTracker, FitnessRecord, RiderProfile	getProgress(), getFTPChange(), getWeightChange(), getFTPGoalProgress(), getWeightGoalProgress()	SD04 partly covers update	Facade, Observer, MVC
F07	Weakness Detection	Hybrid	UC07 Analyze Weaknesses & Improvements	CyclingCoachController, AIChatService, RiderContextService, DeepSeekAdapter, DeepSeekService	analyzeWeaknesses(), buildContext(), sendPrompt(), generateResponse()	SD02 shared AI flow	Facade, Adapter, MVC
F08	Weekly Training Summary	Hybrid	UC08 View Weekly Training Summary	CyclingCoachController, AIChatService, RideHistory, RiderContextService, DeepSeekAdapter, DeepSeekService	getWeeklySummary(), generateWeeklySummary(), buildContext(), sendPrompt(), generateResponse()	SD02 shared AI flow	Facade, Adapter, MVC
F09	Ride Comparison	Deterministic	UC09 Compare Rides	CyclingCoachCLI, CyclingCoachController, RideHistory, RideComparisonService	getRides(), compareRides()	SD05 Compare Two Rides	MVC, Facade
F10	Workout Calendar	Deterministic	UC10 View Workout Calendar	CyclingCoachController, WorkoutCalendarService, TrainingPlan, RideHistory	getWorkoutCalendar(), getWeekWorkouts(), getWeekRides(), update()	SD04 shows Observer mechanism	Observer, Facade, MVC
F11	Personal Best Detection	Deterministic	UC11 View Personal Bests	CyclingCoachController, PersonalBestService, RideHistory	getPersonalBests(), getLongestRide(), getHighestAvgPower(), getHighestMaxPower(), getFastestAvgSpeed()	No dedicated SD	Facade, MVC

4 Explain Each Feature

F01 — Rider Profile Setup

Related Use Case: UC01 — Set Up Rider Profile

Related Sequence Diagram: SD04 — Update Rider Profile

Classes involved: CyclingCoachGUI, CyclingCoachController, RiderProfile, ProgressTracker

Important methods: setUpProfile(), updateProfile(), notifyObservers(), update()

Execution: The cyclist enters their profile information in CyclingCoachGUI and selects save. The GUI sends the profile to CyclingCoachController using setUpProfile(). The controller updates RiderProfile using updateProfile(). After the profile is updated, RiderProfile calls notifyObservers(). ProgressTracker, which is an observer, receives the update through update(). The updated rider information is then stored and can be used by other features.

F02 — Ride Data Import

Related Use Case: UC02 — Import Ride Data

Related Sequence Diagram: SD03 — Import Ride Data

Classes involved: CyclingCoachGUI, CyclingCoachController, CSVImporter, Ride, RideHistory

Important methods: importRide(), addRide()

Execution: The cyclist selects a CSV file in the GUI. The filepath is sent to CyclingCoachController, which calls CSVImporter.importRide(). CSVImporter reads the CSV data and creates a Ride object. The ride is returned to the controller and added into RideHistory using addRide(). The ride can then be used by other features such as AI analysis, weekly summary, comparison, and personal best detection.

F03 — AI Ride Analysis

Related Use Case: UC03 — Analyze Ride

Related Sequence Diagram: Shared AI interaction flow

Classes involved: CyclingCoachGUI, CyclingCoachController, RideAnalysisService, RiderContextService, DeepSeekAdapter, DeepSeekService

Important methods: analyzeRide(), buildRideContext(), sendPrompt(), generateResponse()

Execution: The cyclist selects a completed ride and chooses Analyze Ride. The GUI sends the selected ride to CyclingCoachController, which calls RideAnalysisService.analyzeRide(). RideAnalysisService uses RiderContextService.buildRideContext() to convert the rider profile and selected ride into information that can be used in a prompt. The prompt is sent through DeepSeekAdapter to DeepSeekService. DeepSeek generates the analysis and the response is returned through the controller to the GUI for the cyclist to view.

F04 — Weekly Training Plan Generation

Related Use Case: UC04 — Generate Weekly Training Plan

Related Sequence Diagram: SD01 — Generate Weekly Training Plan

Classes involved: CyclingCoachGUI, CyclingCoachController, TrainingPlanGenerator, TrainingStrategy, RiderContextService, DeepSeekAdapter, DeepSeekService, TrainingPlanParser, TrainingPlan, WorkoutCalendarService

Important methods: generateTrainingPlan(), generatePlan(), buildPrompt(), buildContext(), sendPrompt(), generateResponse(), parsePlan(), notifyObservers(), update()

Execution: When the cyclist selects Generate Weekly Training Plan, the GUI sends the request to CyclingCoachController. The controller calls TrainingPlanGenerator.generatePlan(). Based on the rider's main goal, the generator uses the correct TrainingStrategy and calls buildPrompt(). RiderContextService also prepares the rider profile and ride history. These two parts are combined and sent through DeepSeekAdapter to DeepSeekService. DeepSeek returns the generated plan, and TrainingPlanParser converts the response into a TrainingPlan containing workouts. After the plan changes, TrainingPlan notifies its observers and WorkoutCalendarService.update() refreshes the calendar.

F05 — AI Chat

Related Use Case: UC05 — Ask AI Question

Related Sequence Diagram: SD02 — Ask AI Question

Classes involved: CyclingCoachGUI, CyclingCoachController, AIChatService, RiderContextService, DeepSeekAdapter, DeepSeekService

Important methods: askAI(), askQuestion(), buildContext(), sendPrompt(), generateResponse()

Execution: The cyclist enters a question in the AI chat. The GUI sends the question to CyclingCoachController.askAI(). The controller calls AIChatService.askQuestion(). RiderContextService prepares the rider profile and ride history, and this information is combined with the cyclist's question. The prompt is sent through DeepSeekAdapter to DeepSeekService. DeepSeek returns a response and the response is returned through the controller to the GUI.

F06 — Progress & Goal Tracking

Related Use Case: UC06 — View Progress & Goal Tracking

Related Sequence Diagram: SD04 partly covers the update process

Classes involved: CyclingCoachController, ProgressTracker, FitnessRecord, RiderProfile

Important methods: getProgress(), getFTPChange(), getWeightChange(), getFTPGoalProgress(), getWeightGoalProgress()

Execution: When the cyclist opens Progress & Goal Tracking, the request is sent to CyclingCoachController. The controller gets the progress information from ProgressTracker. ProgressTracker uses the stored FitnessRecord objects to calculate changes in FTP and weight. It also compares the current FTP and weight with the target values in RiderProfile. The results are returned to the controller and displayed to the cyclist.

F07 — Weakness / Improvement Detection

Related Use Case: UC07 — Analyze Weakness

Related Sequence Diagram: SD02 — Shared AI flow

Classes involved: CyclingCoachController, AIChatService, RiderContextService, DeepSeekAdapter, DeepSeekService

Important methods: analyzeWeaknesses(), buildContext(), sendPrompt(), generateResponse()

Execution: The cyclist selects Analyze Weakness. The request is sent to CyclingCoachController, which calls AIChatService.analyzeWeaknesses(). RiderContextService prepares the rider profile and ride history. The rider context is combined with a predefined prompt for weakness analysis. The prompt is sent

through DeepSeekAdapter to DeepSeekService. DeepSeek analyzes the rider information and returns a response showing weaknesses and possible areas for improvement. The result is then displayed to the cyclist.

F08 — Weekly Training Summary

Related Use Case: UC08 — View Weekly Training Summary

Related Sequence Diagram: SD02 — Shared AI flow

Classes involved: CyclingCoachController, AIChatService, RideHistory, RiderContextService, DeepSeekAdapter, DeepSeekService

Important methods: getWeeklySummary(), generateWeeklySummary(), buildContext(), sendPrompt(), generateResponse()

Execution: The cyclist selects Weekly Training Summary. CyclingCoachController calls AIChatService.generateWeeklySummary(). The system uses the recent rides from RideHistory, and RiderContextService prepares the rider information. This information is combined with a predefined weekly summary prompt and sent through DeepSeekAdapter to DeepSeekService. DeepSeek returns a summary of the cyclist's recent week of training, which is returned to the controller and displayed by the GUI.

F09 — Ride Comparison

Related Use Case: UC09 — Compare Rides

Related Sequence Diagram: SD05 — Compare Two Rides

Classes involved: CyclingCoachGUI or CyclingCoachCLI, CyclingCoachController, RideHistory, RideComparisonService, Ride

Important methods: getRides(), compareRides()

Execution: The cyclist opens RideHistory and selects two rides to compare. The selected Ride objects are sent through the GUI or CLI to CyclingCoachController. The controller calls RideComparisonService.compareRides(). The service compares values such as distance, duration, average speed, heart rate, and power. The comparison result is returned to the controller and displayed to the cyclist.

F10 — Workout Calendar

Related Use Case: UC10 — View Workout Calendar

Related Sequence Diagram: SD04 demonstrates the Observer update mechanism

Classes involved: CyclingCoachController, WorkoutCalendarService, TrainingPlan, RideHistory

Important methods: getWorkoutCalendar(), getWeekWorkouts(), getWeekRides(), update()

Execution: When the cyclist opens the Workout Calendar, CyclingCoachController calls WorkoutCalendarService. The service uses getWeekWorkouts() to get planned workouts from TrainingPlan and getWeekRides() to get completed rides from RideHistory. These are combined into the current week's calendar.

WorkoutCalendarService is also an observer of TrainingPlan, so when the training plan changes, update() can refresh the calendar automatically.

F11 — Personal Best Detection

Related Use Case: UC11 — View Personal Bests

Related Sequence Diagram: No dedicated sequence diagram

Classes involved: CyclingCoachController, PersonalBestService, RideHistory, Ride

Important methods: getPersonalBests(), getLongestRide(), getHighestAvgPower(), getHighestMaxPower(), getFastestAvgSpeed()

Execution: When the cyclist selects Personal Bests, the GUI sends the request to CyclingCoachController. The controller calls PersonalBestService, which reads the rides stored in RideHistory. The service checks the ride data to find records such as longest ride, highest average power, highest maximum power, and fastest average speed. The results are returned to the controller and displayed to the cyclist.